

Docker

para Devs

Do conceito a prática com analogias do nosso mundo

Do zero ao container

3 Exercícios Práticos

Cheat Sheet

Sumário

01	O que é Docker?	2
02	Pré-requisito: entendendo Containers e VMs	3
03	Para que serve o Docker?	4
04	Instalando o Docker	5
05	Conceitos essenciais	6
06	Exercício 1 — Ola, Container! [Iniciante]	8
07	Exercício 2 — Dockerizando sua app [Intermediario]	9
08	Exercício 3 — App + Banco juntos [Desafio]	11
09	Cheat Sheet	13
10	Próximos passos	14

Como usar este mini book

Leia do começo ao fim na primeira vez.

Os capítulos foram pensados em sequência: cada um prepara o terreno para o seguinte. Se você pular direto para os exercícios sem ler os conceitos, pode ficar perdida em alguns comandos.

Volte sempre que precisar.

O Cheat Sheet da página 13 foi feito para ficar aberto enquanto você pratica. Deixe ele no segundo monitor ou impresso do lado do computador.

Faca os exercícios de verdade.

Ler sobre Docker ensina a teoria. Rodar os comandos é que fixa de verdade. Abra o terminal e execute cada passo. Não tem atalho aqui.

01 • O que é Docker?

Docker é uma plataforma open source que permite criar, distribuir e executar aplicações dentro de **containers**. Sabe aquela situação chata que toda dev já viveu? Você passa horas desenvolvendo, tudo funciona lindo na sua máquina, aí manda pra colega ou sobe em produção e... simplesmente não roda.

O Docker chegou pra acabar com isso de vez. Ele empacota a aplicação junto com absolutamente tudo que ela precisa pra funcionar: bibliotecas, variáveis de ambiente, dependências e configurações. Esse pacote é o **container**, e ele roda igual em qualquer lugar.

>> Imagina que você fez um bolo de chocolate incrível. Em vez de só passar a receita pra sua amiga, você entrega o bolo já montado, com todos os ingredientes medidos, na caixinha certa, prontinho pra servir. Não importa se a cozinha dela é diferente da sua, o bolo vai ficar igual. O Docker faz exatamente isso com a sua aplicação: entrega o bolo pronto, não só a receita.

[DICA] Por que isso importa pra você?

Hoje em dia as aplicações precisam rodar em vários lugares ao mesmo tempo: no computador da dev, no servidor de testes e em produção na nuvem. Sem o Docker, cada um desses ambientes precisaria ser configurado na mão. Qualquer diferençazinha vira um bug misterioso que ninguém consegue reproduzir. Com o Docker, todo mundo roda exatamente a mesma coisa, sem surpresa.

02 • Pré-requisito: Containers e VMs

Antes de mergulhar de cabeça no Docker, vale a pena entender dois conceitos que ele usa como base: as **máquinas virtuais (VMs)** e os **containers**. Se você já conhece esses dois, fique à vontade para pular direto pro capítulo 03. Mas se ainda tem dúvida, fica aqui que esse resumo vai clarear tudo.

[LIVRO] Quer aprofundar mais?

O que você vai ler aqui é um resumo introdutório para você não ficar perdida. O Volume 02 da Série Deploy do Zero, sobre Git e GitHub, e materiais de Sistemas Operacionais cobrem esses fundamentos com muito mais profundidade. Vale a leitura para construir uma base mais sólida!

Máquina Virtual (VM)

Uma VM simula um computador inteiro dentro do seu computador. Ela tem seu próprio sistema operacional, memória e disco, tudo virtualizado. Pensa assim: é como morar num apartamento dentro de um prédio. O prédio é o servidor físico real, e o apartamento é a VM rodando lá dentro.

>> *Imagina uma maleta de maquiagem gigante, daquelas profissionais, com todos os compartimentos, espelhos embutidos, luzes próprias e até carregador de bateria. Ela é completa e autossuficiente, mas pesada demais pra carregar no dia a dia. Uma VM é assim: funciona sozinha pra tudo, mas consome muito recurso da máquina.*

Container

O container é uma abordagem muito mais leve. Em vez de virtualizar o sistema operacional inteiro, ele compartilha o kernel, que é o núcleo do SO, do computador host. Ele isola apenas o que a aplicação precisa, sem carregar todo o resto.

>> *Agora pensa numa nécessaire compacta que você monta para aquele evento específico: só o batom, a base e a máscara de cílios que você vai usar. Leve, fácil de pegar e perfeita pra aquele momento. O container é exatamente isso: enxuto, rápido e focado no que importa.*

VM x Container: comparativo rápido

Critério	VM	Container
Tamanho	Gigabytes	Megabytes
Inicialização	Minutos	Segundos
SO próprio?	Sim	Não (compartilha)
Portabilidade	Média	Alta
Consumo de RAM	Alto	Baixo

03 • Para que serve o Docker?

Beleza, você já sabe o que é o Docker. Mas onde ele entra na prática? Onde você vai encontrar ele no seu dia a dia como dev? Vem que eu te mostro os casos de uso mais comuns.

Consistencia entre ambientes

Essa é a dor que o Docker resolve melhor. Sabe o famoso *na minha máquina funciona*? Com o Docker isso deixa de existir. O container garante que o ambiente de desenvolvimento, o de testes e o de produção rodam a mesma coisa, do mesmo jeito, sempre.

Deploy mais rápido

Containers sobem em questão de segundos. Isso transforma completamente o processo de atualizar uma aplicação em produção. Menos tempo esperando, mais tempo entregando valor.

Microserviços

Cada parte da aplicação pode rodar no seu próprio container, com a sua própria versão de linguagem e dependências, sem conflito com as outras partes. É o modelo ideal para arquiteturas modernas.

>> Pensa num look completo montado pra um evento. O vestido, os brincos, a bolsa e os sapatos podem ter vindo de lojas completamente diferentes, mas você combinou tudo com perfeição. No Docker é igual: cada container é uma peça independente, mas juntos formam a aplicação completa funcionando lindamente.

Portabilidade para a nuvem

Um container que funciona na sua máquina vai funcionar igualzinho na AWS, no Azure ou no Google Cloud. Sem precisar reconfigurar nada, sem aquele susto na hora do deploy.

Ambientes de teste isolados

Precisa testar a aplicação com uma versão diferente do banco de dados? Sobe um container, testa tudo que precisar e depois apaga. O seu sistema fica intacto, sem nada instalado à toa.

[STAT] Docker no mercado

Segundo a pesquisa Stack Overflow Developer Survey, Docker está entre as plataformas mais amadas e usadas por devs no mundo todo. Dominar Docker hoje é habilidade essencial para qualquer vaga backend, DevOps ou cloud.

04 • Instalando o Docker

Antes de colocar a mão na massa, precisamos instalar o Docker na sua máquina. O processo é bem tranquilo e varia um pouquinho dependendo do seu sistema operacional. Escolha o seu caso abaixo:

Windows e Mac

A forma mais simples é instalar o **Docker Desktop**, que já vem com interface visual é tudo que você precisa pra começar:

- Acesse <https://www.docker.com/products/docker-desktop>
- Baixe a versão para o seu sistema operacional
- Siga o instalador normalmente até o final
- Abra o Docker Desktop e aguarde ele inicializar completamente

Linux (Ubuntu ou Debian)

Abra o terminal e rode os comandos abaixo em sequência:

Terminal Linux

```
# Atualizar pacotes
sudo apt-get update
# Instalar o Docker
sudo apt-get install docker.io -y
# Iniciar o serviço
sudo systemctl start docker
sudo systemctl enable docker
# Adicionar seu usuário ao grupo docker (evita usar sudo)
sudo usermod -aG docker $USER
```

Verificando se deu certo

Abra o terminal e rode esses dois comandos. Se tudo estiver ok, você já está pronta pra começar:

Terminal

```
docker --version
# Esperado: Docker version 24.x.x
docker run hello-world
# Se aparecer Hello from Docker! está tudo certo!
```

[!] Atenção usuarias Windows

Certifique-se de que o WSL 2 (Windows Subsystem for Linux) está ativado na sua máquina. O próprio Docker Desktop vai te guiar nessa configuração caso ainda não esteja ativo. É o único pré-requisito técnico no Windows e o processo é bem simples.

05 · Conceitos essenciais

Antes de ir pra parte prática, vamos alinhar o vocabulário. Esses são os termos que você vai ver o tempo todo quando trabalhar com Docker. Não precisa decorar tudo agora, mas vale ter essa página marcada pra consultar sempre que precisar.

Image (Imagem)

Definição: É o modelo, o molde do container. Ela é imutável e pode ser reutilizada quantas vezes você quiser.

>> Pensa na receita completa do bolo, com todos os ingredientes e o modo de preparo. Você pode usar a mesma receita para fazer varios bolos. A imagem é essa receita e cada bolo que sai dela é um container.

Container

Definição: É a imagem em execução. Uma instância ativa e isolada da sua aplicação rodando na sua máquina.

>> O bolo já assado, montado e pronto pra servir. Você pode ter varios bolos feitos da mesma receita ao mesmo tempo, cada um num pratinho diferente.

Dockerfile

Definição: Um arquivo de texto com as instruções para construir uma imagem. É o ponto de partida de tudo.

>> A sua caderneta de receitas escrita do zero, passo a passo: pre-aqueça o forno, misture os ingredientes, asse por 40 minutos. O Docker lê esse arquivo e cria sua imagem seguindo cada instrução.

Docker Hub

Definição: Um repositório público cheio de imagens prontas pra usar. Funciona como um GitHub, mas para imagens Docker.

>> A loja de insumos de confeitaria online. Você não precisa criar tudo do zero: já tem imagem pronta do PostgreSQL, Redis, Nginx é muito mais. É só baixar e usar.

Port Mapping (Portas)

Definição: Mapeia uma porta de dentro do container para uma porta da sua máquina, deixando a aplicação acessível.

>> *Pensa no interfone do seu apartamento. O container tem a porta dele lá dentro, mas você configura a entrada pelo portão do prédio para o mundo externo conseguir chegar até ele.*

Volume

Definição: Mecanismo para persistir dados do container no disco da máquina host.

>> *Como aquela pasta de arquivos na nuvem que você sincroniza em qualquer dispositivo. O que você salvar no volume persiste mesmo que o container seja removido. Sem volume, quando o container morre, os dados somem junto.*

Exemplos de mapeamento de portas

Comando	O que faz
-p 8080:80	Acesse localhost:8080 para chegar na porta 80 do container
-p 5432:5432	Banco PostgreSQL acessível na sua máquina normalmente
-p 3000:3000	App Node.js rodando no container, acesso pela porta 3000
-p 443:443	HTTPS direto, sem tradução de porta

[DICA] Resumo rápido antes de praticar

Image é o molde. Container é o molde rodando. Dockerfile é o arquivo que cria o molde. Docker Hub é onde os moldes ficam guardados. Volume salva seus dados. Porta é o canal de comunicação com o mundo externo. Agora você já tem o vocabulário completo para os exercícios!

Exercício 1 — Ola, Container!

INICIANTE

Objetivo: Rodar seu primeiro container Docker e entender o fluxo básico de imagem para container.

Tempo estimado: 10 minutos

O que você vai aprender

- Como baixar uma imagem do Docker Hub
- Como executar um container com o comando **docker run**
- Como listar containers rodando e parados
- Como parar e remover containers

Passo a passo

1. Confirme que o Docker está instalado e funcionando:

```
docker --version
```

2. Rode o container hello-world, o clássico Olá Mundo do Docker:

```
docker run hello-world
# 0 Docker vai fazer quatro coisas na sequência:
# 1. Procurar a imagem hello-world na sua máquina
# 2. Nao encontrar (e a primeira vez!)
# 3. Baixar automaticamente do Docker Hub
# 4. Criar e rodar o container
```

3. Sobe um servidor web Nginx rodando em segundo plano:

```
docker run -d -p 8080:80 --name meu-nginx nginx
# -d      = roda em background, sem travar o terminal
# -p      = mapeia a porta: sua_máquina:container
# --name  = da um nome amigavel pro container
```

Abra o navegador em **http://localhost:8080**. A página padrão do Nginx vai aparecer.

4. Pratique os comandos de gerenciamento:

```
docker ps          # lista containers rodando agora
docker ps -a       # lista todos, ate os parados
docker stop meu-nginx # para o container
docker rm meu-nginx  # remove o container
docker images      # lista as imagens baixadas
```

[OK] Resultado esperado

Você baixou uma imagem, subiu um container, acessou pelo navegador e praticou os comandos essenciais. Se o Nginx apareceu no browser, você completou o exercício com sucesso!

Exercício 2 — Dockerizando sua App

INTERMEDIÁRIO

Objetivo: Criar um Dockerfile para uma aplicação Python e construir sua própria imagem Docker do zero.

Tempo estimado: 20 a 30 minutos

O que você vai aprender

- Como escrever um **Dockerfile** do zero
- Como construir uma imagem com o comando **docker build**
- Como rodar um container a partir da sua própria imagem
- Boas práticas na hora de escrever um Dockerfile

Vamos criar uma pequena API em Python usando Flask. Ela vai retornar uma mensagem personalizada com o seu nome quando acessada pelo navegador.

Passo a passo

1. Crie uma pasta para o projeto e entre nela:

```
mkdir minha-api-docker
cd minha-api-docker
```

2. Crie o arquivo **app.py** com o seguinte conteúdo:

```
app.py
# app.py
from flask import Flask
import os
app = Flask(__name__)
@app.route('/')
def home():
    nome = os.getenv('NOME_DEV', 'Dev')
    return f'Oi, {nome}! Seu container está rodando com sucesso'
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

3. Crie o arquivo **requirements.txt**:

```
requirements.txt
flask==3.0.0
```

4. Crie o **Dockerfile** (sem nenhuma extensão no nome):

```
Dockerfile
FROM python:3.11-slim
WORKDIR /app
# Copiamos as dependências primeiro para aproveitar o cache
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ['python', 'app.py']
```

5. Faça o build e suba o container:

```
# Constroi a imagem (o ponto indica a pasta atual)
docker build -t minha-api:v1 .
# Sobe o container passando seu nome como variavel
docker run -d -p 5000:5000 -e NOME_DEV='Gabi' --name api minha-api:v1
```

Acesse **http://localhost:5000** no navegador e veja sua mensagem personalizada.

[DICA] Boas práticas no Dockerfile

Sempre copie o arquivo de dependências (requirements.txt ou package.json) ANTES de copiar o restante do código. O Docker usa um sistema de cache por camadas: se as dependências não mudaram, ele reutiliza a camada já construída e o build fica muito mais rápido. Só copia o código novo quando algo realmente mudou.

Comandos uteis durante o desenvolvimento

1. Ver o que está acontecendo dentro do container:

```
docker logs -f api          # acompanha logs em tempo real
docker exec -it api bash    # abre terminal dentro do container
```

2. Recriar o container depois de mudar o código:

```
docker rm -f api
docker build -t minha-api:v1 .
docker run -d -p 5000:5000 -e NOME_DEV='Gabi' --name api minha-api:v1
```

[OK] Resultado esperado

Sua imagem aparece na listagem do docker images, o container sobe sem erros e o navegador exibe a mensagem com o seu nome. Você acabou de criar sua primeira imagem Docker personalizada!

Exercício 3 — App + Banco com Docker Compose

DESAFIO

Objetivo: Orquestrar dois containers ao mesmo tempo, a API e um banco de dados, usando o Docker Compose.

Tempo estimado: 30 a 40 minutos

[DICA] Spoiler do próximo volume!

Docker Compose é o tema completo do Volume 04 da série. Aqui você vai ter um gostinho do que ele faz, o suficiente para já sair usando no dia a dia. Para dominar de verdade, não perca o próximo mini book!

O que você vai aprender

- O que é e para que serve o arquivo **docker-compose.yml**
- Como conectar containers por uma rede interna
- Como passar variáveis de ambiente para configurar os serviços
- Como subir tudo de uma vez com um único comando

Vamos expandir a API do exercício anterior para que ela se conecte a um banco de dados PostgreSQL.

Passo a passo

1. Atualize o **requirements.txt**:

```
requirements.txt
flask==3.0.0
psycopg2-binary==2.9.9
sqlalchemy==2.0.23
```

2. Crie o arquivo **docker-compose.yml** — bloco da API:

docker-compose.yml (parte 1)

```
version: '3.8'
services:
  api:
    build: .
    ports:
      - '5000:5000'
    environment:
      - NOME_DEV=Gabi
      - DB_HOST=banco
    depends_on:
      - banco
    networks:
      - rede-app
```

. Continuação — bloco do banco e volumes:

docker-compose.yml (parte 2)

```
banco:
  image: postgres:15-alpine
  environment:
    - POSTGRES_USER=postgres
    - POSTGRES_PASSWORD=minhasenha
    - POSTGRES_DB=appdb
  volumes:
    - dados-banco:/var/lib/postgresql/data
  networks:
    - rede-app
volumes:
  dados-banco:
networks:
  rede-app:
```

3. Suba toda a aplicação com um único comando:

```
docker compose up -d
docker compose logs -f    # acompanha os logs
docker compose down       # para tudo
```

>> Lembra quando você montou um look completo para aquele evento especial? Vestido, brincos, bolsa e sapato, cada peça no lugar certo, todas combinando entre si. O **docker-compose.yml** faz exatamente isso com a sua aplicação: descreve cada serviço, como eles se conectam e garante que tudo suba junto.

[OK] Resultado esperado

Com um único `docker compose up`, sua API e o banco sobem juntos e já se enxergam na rede interna. A API fala com o banco usando apenas o nome banco, sem precisar saber o IP de nada. Os dados persistem no volume mesmo que o container seja reiniciado.

09 • Cheat Sheet de Comandos

Cola essa página na parede ou salva nos favoritos. São os comandos que você vai usar toda semana quando trabalhar com Docker:

Imagens	
<code>docker images</code>	Lista todas as imagens locais
<code>docker pull nginx</code>	Baixa uma imagem do Docker Hub
<code>docker build -t nome:tag .</code>	Constroi imagem a partir do Dockerfile
<code>docker rmi nome:tag</code>	Remove uma imagem
<code>docker image prune</code>	Remove imagens que não estão sendo usadas
Containers	
<code>docker run -d -p 8080:80 nginx</code>	Sobe container em background com porta
<code>docker run -it ubuntu bash</code>	Sobe container interativo com terminal
<code>docker ps</code>	Lista containers rodando agora
<code>docker ps -a</code>	Lista todos os containers
<code>docker stop id</code>	Para um container
<code>docker rm id</code>	Remove um container parado
<code>docker rm -f id</code>	Para e remove de uma vez
Inspeção e Logs	
<code>docker logs id</code>	Vê os logs do container
<code>docker logs -f id</code>	Acompanha os logs em tempo real
<code>docker exec -it id bash</code>	Abre terminal dentro do container
<code>docker inspect id</code>	Mostra todos os detalhes do container
<code>docker stats</code>	Monitora CPU e memória em tempo real

Limpeza	
<code>docker system prune</code>	Remove tudo que não está em uso
<code>docker volume ls</code>	Lista os volumes existentes
<code>docker volume prune</code>	Remove volumes não utilizados
<code>docker network ls</code>	Lista as redes criadas

10 • Próximos Passos

Parabéns por chegar até aqui! Você agora tem uma base sólida de Docker e já consegue usar no seu dia a dia. Mas a jornada está longe de acabar. Olha o que vem adiante na série:

Vol. 04 — Docker Compose

Você já viu um gostinho no exercício 3. O próximo mini book mergulha de verdade no Compose: gerenciamento de múltiplos serviços, redes customizadas, healthchecks e como montar um ambiente de desenvolvimento completo.

Vol. 05 — APIs REST com Python

Aprenda a construir APIs completas usando Flask ou FastAPI, com boas práticas de REST, autenticação e documentação. Tudo dockerizado, claro.

Vol. 06 — AWS na Prática

Com Docker dominado, a nuvem fica muito mais tranquila. Vamos ao Elastic Container Service (ECS), ECR para hospedar suas imagens, é muito mais.

Recursos para continuar estudando

- **Documentação oficial:** docs.docker.com — sempre a fonte mais atualizada e confiável
- **Docker Hub:** hub.docker.com — explore as imagens prontas disponíveis
- **Play with Docker:** labs.play-with-docker.com — pratique direto no browser sem instalar nada
- **Livro:** Docker Deep Dive de Nigel Poulton — leitura rápida, direta é muito prática

O que você dominou neste volume

- O que é Docker e por que ele importa no mercado
- A diferença entre VM e container e quando usar cada um
- Os 6 conceitos essenciais: Image, Container, Dockerfile, Docker Hub, Port Mapping e Volume
- Como rodar containers, baixar imagens e gerenciar o ciclo de vida
- Como escrever um Dockerfile do zero e construir sua própria imagem
- Como orquestrar API + banco de dados com Docker Compose

Você chegou até aqui. Isso já te coloca na frente de muita gente.

Docker não é só uma ferramenta. É uma forma de pensar sobre ambientes, reprodutibilidade e cuidado com o código que você entrega.

Gostou desse mini book?

Compartilha com aquela amiga dev que ta começando a jornada!

Me marca no Instagram **@deployada** com o seu progresso.

Adoro acompanhar vocês evoluindo!

Instagram	LinkedIn	Newsletter
@deployada	linkedin.com/in/mgcarvalho	deployada

Continue sua jornada! >>

Vol. 04 • Docker Compose — orquestrando containers

Vol. 05 • APIs REST com Python

Vol. 06 • AWS na prática: S3, EC2 e Lambda

Vol. 07 • Mensageria: SQS & SNS

Vol. 08 • Observabilidade: Datadog & Splunk

"Toda grande dev um dia rodou seu primeiro container."

— @deployada

@deployada

Instagram • LinkedIn • Newsletter